

تحليل جامع پروژه Polaris-Client

علی صمدی فرد - نیوشا یقینی - مهدی اوشنی

۲۲ مرداد ۱۴۰۴

فهرست مطالب

۲	اول شرح کلی پروژه
۳	۱ معرفی و اهداف پروژه
۳	۱.۱ چرا این پروژه توسعه یافته است؟
۳	۲.۱ این اپلیکیشن برای چه کاری استفاده می‌شود؟
۵	دوم مشخصات و ویژگی‌های نرم‌افزار
۶	۲ تحلیل فنی و امکانات
۶	۱.۲ مشخصات فنی و کتابخانه‌ها
۷	۲.۲ ویژگی‌ها و امکانات اصلی
۹	سوم بررسی و تحلیل کد
۱۰	۳ ساختار پروژه و فایل‌ها
۱۰	۱.۳ پوشه controllers - مدیریت سرویس‌ها و وظایف
۱۰	۱.۱.۳ فایل ServiceManager.kt
۱۱	۲.۱.۳ فایل‌های *Worker.kt (مانند PingTestWorker.kt)
۱۲	۳.۱.۳ فایل LocationService.kt و CellularService.kt
۱۲	۲.۳ پوشه views - رابط کاربری و منطق نمایش
۱۲	۱.۲.۳ فایل MainActivity.kt
۱۲	۲.۲.۳ فایل MapActivity.kt
۱۳	۳.۲.۳ فایل‌های فرگمنت تست (مانند PingTestFragment.kt)
۱۳	۳.۳ پوشه utils - ابزارهای کمکی
۱۳	۱.۳.۳ فایل DatabaseHelper.kt
۱۴	۲.۳.۳ فایل TokenManager.kt
۱۴	۴.۳ پوشه models - ساختارهای داده

بخش اول

شرح کلی پروژه

فصل ۱

معرفی و اهداف پروژه

پروژه **Polaris-Client** یک اپلیکیشن **Drive Test** اندرویدی است که به عنوان بخشی از پلتفرم بزرگتر **Polaris** توسعه یافته است. هدف اصلی این پلتفرم، نظارت بر وضعیت اینترنت، شناسایی فیلترینگ و سنجش کیفیت و عملکرد شبکه در مقیاس گسترده و به روش جمع‌سپاری (Crowdsourcing) است.

۱.۱ چرا این پروژه توسعه یافته است؟

در دنیای امروز، دسترسی آزاد و با کیفیت به اینترنت یکی از ارکان اصلی ارتباطات، آموزش و اقتصاد است. با این حال، در بسیاری از نقاط جهان، کاربران با مشکلاتی نظیر سانسور، فیلترینگ، کیفیت پایین شبکه و قطعی‌های مکرر مواجه هستند. تشخیص و اثبات این مشکلات به صورت متمرکز بسیار دشوار است، زیرا وضعیت شبکه می‌تواند از یک منطقه به منطقه دیگر، از یک اپراتور به اپراتور دیگر و حتی در ساعات مختلف شبانه‌روز متغیر باشد.

پروژه **Polaris** برای حل این چالش طراحی شده است. این پلتفرم با استفاده از یک اپلیکیشن کلاینت (همین پروژه) که توسط کاربران داوطلب نصب می‌شود، داده‌های دقیقی از وضعیت شبکه در نقاط جغرافیایی مختلف جمع‌آوری می‌کند. این داده‌ها به سرور مرکزی ارسال شده و پس از تحلیل، به شکل نقشه و گزارش‌های آماری نمایش داده می‌شوند. این اطلاعات می‌تواند برای محققان، روزنامه‌نگاران، فعالان حقوق دیجیتال و حتی خود کاربران بسیار ارزشمند باشد تا تصویری واقعی از وضعیت اینترنت در مناطق مختلف به دست آورند.

۲.۱ این اپلیکیشن برای چه کاری استفاده می‌شود؟

کاربر با نصب اپلیکیشن **Polaris-Client**، به این پروژه اجازه می‌دهد تا به دو صورت **فعال** و **غیرفعال** به جمع‌آوری داده بپردازد:

- **جمع‌آوری فعال (Active):** کاربر می‌تواند به صورت دستی تست‌های مختلفی مانند تست پینگ، تست سرعت دانلود و آپلود، تست DNS و تست HTTP را اجرا کند و نتایج آن را مشاهده نماید.
- **جمع‌آوری غیرفعال (Passive):** این بخش مهم‌ترین قسمت پروژه است. اپلیکیشن به صورت دوره‌ای و در پس‌زمینه (Background)، بدون نیاز به دخالت کاربر، تست‌های مشخصی را اجرا می‌کند. همزمان با اجرای تست‌ها، اطلاعات مهم دیگری مانند موقعیت جغرافیایی (GPS)، نوع شبکه (Wi-Fi, 4G, 5G)، قدرت سیگنال و اطلاعات اپراتور تلفن همراه نیز جمع‌آوری می‌شود.

تمام داده‌های جمع‌آوری شده به صورت ناشناس (Anonymous) به سرور ارسال می‌گردد تا حریم خصوصی کاربران حفظ شود. در نهایت، این داده‌ها به ساخت یک نقشه جامع از کیفیت و وضعیت سانسور اینترنت کمک می‌کنند.

بخش دوم

مشخصات و ویژگی های نرم افزار

فصل ۲

تحلیل فنی و امکانات

این بخش به بررسی ویژگی‌های فنی، کتابخانه‌های استفاده شده و امکانات اصلی اپلیکیشن Polaris-Client می‌پردازد.

۱.۲ مشخصات فنی و کتابخانه‌ها

با بررسی فایل `app/build.gradle.kts` و سایر فایل‌های پیکربندی، می‌توان به مشخصات فنی زیر اشاره کرد:

- زبان برنامه‌نویسی: کاتلین (Kotlin)
- معماری: استفاده از معماری مدرن اندروید با کامپوننت‌هایی مانند `ViewModel`، `LiveData` و `Navigation Component`.
- رابط کاربری (UI): مبتنی بر `View` های `XML` و استفاده از `Material Design Components`.
- کتابخانه‌های اصلی:
 - `androidx.core:core-ktx`: برای ساده‌سازی کدهای کاتلین در اندروید.
 - `androidx.appcompat:appcompat`: برای پشتیبانی از ویژگی‌های جدید در نسخه‌های قدیمی‌تر اندروید.
 - `com.google.android.material:material`: برای استفاده از کامپوننت‌های طراحی متریال.
 - `androidx.lifecycle`: برای مدیریت چرخه حیات کامپوننت‌های `UI` (`ViewModel`، `LiveData`).
 - `androidx.navigation`: برای مدیریت ناوبری بین صفحات و فرگمنت‌ها.
 - `androidx.work:work-runtime-ktx`: کتابخانه **WorkManager** برای مدیریت وظایف پس‌زمینه به صورت قابل اطمینان و بهینه. این کتابخانه نقش کلیدی در اجرای تست‌ها به صورت غیرفعال دارد.
 - `com.google.android.gms:play-services-maps`: برای نمایش نقشه گوگل و داده‌های جمع‌آوری شده روی آن.
 - `com.google.android.gms:play-services-location`: برای دریافت موقعیت مکانی کاربر.
 - `io.ktor:ktor-client-*`: یک فریم‌ورک قدرتمند برای انجام درخواست‌های شبکه (HTTP Client) که برای ارسال داده‌ها به سرور استفاده می‌شود.
 - `androidx.sqlite:sqlite-ktx`: برای کار با پایگاه داده محلی `SQLite` به منظور ذخیره نتایج تست‌ها قبل از ارسال به سرور.

۲.۲ ویژگی‌ها و امکانات اصلی

اپلیکیشن دارای مجموعه امکانات جامعی برای سنجش شبکه است:

۱. احراز هویت کاربر:

- صفحه ورود (Login) و ثبت نام (Signup).
- مدیریت توکن (Token) برای حفظ وضعیت ورود کاربر در اپلیکیشن.

۲. داشبورد و صفحه اصلی:

- نمایش اطلاعات کلی شبکه و وضعیت سرویس پس زمینه.
- امکان فعال/غیرفعال کردن سرویس جمع‌آوری داده در پس زمینه.

۳. مجموعه تست‌های شبکه (فعال و غیرفعال):

- تست **Ping**: اندازه‌گیری تأخیر (Latency) در ارسال بسته به یک سرور مشخص.
- تست **DNS**: بررسی زمان پاسخ‌دهی سرور DNS و صحت آدرس IP بازگردانده شده برای یک دامنه. این تست برای تشخیص DNS Hijacking کاربرد دارد.
- تست **HTTP**: ارسال درخواست HTTP به یک آدرس و بررسی کد وضعیت (Status Code) و محتوای پاسخ برای تشخیص فیلترینگ در لایه HTTP.
- تست **سرعت (Speed Test)**: اندازه‌گیری سرعت دانلود و آپلود با استفاده از یک سرور تست.
- تست **پیامک (SMS Test)**: ارسال یک پیامک به یک شماره خاص و انتظار برای دریافت پاسخ جهت بررسی تأخیر و صحت عملکرد سرویس پیامک. (این ویژگی نیاز به دسترسی‌های حساس دارد).

۴. جمع‌آوری داده در پس زمینه:

- استفاده از **WorkManager** برای زمان‌بندی و اجرای تست‌ها به صورت دوره‌ای (مثلاً هر چند ساعت یک‌بار).
- اجرای سرویس‌ها حتی در صورتی که اپلیکیشن بسته باشد یا دستگاه ری‌استارت شود.
- بهینه‌سازی مصرف باتری با اجرای وظایف در شرایط مناسب (مثلاً هنگام اتصال به Wi-Fi یا در حال شارژ بودن دستگاه).

۵. نمایش داده‌ها روی نقشه:

- یک نقشه تعاملی که نتایج تست‌های جمع‌آوری شده از کاربران مختلف را بر اساس موقعیت جغرافیایی نمایش می‌دهد.
- استفاده از مارکرهای رنگی برای نمایش کیفیت شبکه در نقاط مختلف (مثلاً سبز برای کیفیت خوب، قرمز برای کیفیت بد).
- نمایش جزئیات هر تست با کلیک روی مارکر مربوطه.

۶. ذخیره‌سازی محلی:

- نتایج تمام تست‌ها ابتدا در یک پایگاه داده SQLite روی دستگاه ذخیره می‌شوند.

- این کار باعث می‌شود در صورت قطع بودن اینترنت، داده‌ها از بین نروند و در فرصت مناسب به سرور ارسال شوند.

۷. مدیریت تم (Theme):

- پشتیبانی از حالت تاریک (Dark Mode) و روشن (Light Mode) و هماهنگی با تنظیمات سیستم عامل.

بخش سوم

بررسی و تحلیل کد

فصل ۳

ساختار پروژه و فایل‌ها

در این بخش، فایل‌های کلیدی پروژه به تفکیک ساختار پوشه‌ها بررسی می‌شوند. تمرکز بر روی فایل‌های کاتلین (.kt) است که منطق اصلی اپلیکیشن را تشکیل می‌دهند.

۱.۳ پوشه controllers - مدیریت سرویس‌ها و وظایف

این بسته (package) قلب تپنده اپلیکیشن در زمینه جمع‌آوری داده است. کلاس‌های این بخش مسئولیت اجرای تست‌ها در پس‌زمینه و مدیریت سرویس‌ها را بر عهده دارند.

۱.۱.۳ فایل ServiceManager.kt

- هدف: این کلاس وظیفه مدیریت کلی سرویس پس‌زمینه را دارد. این کلاس یک نقطه مرکزی برای شروع و توقف جمع‌آوری داده‌های غیرفعال است.

- توضیح کد: این کلاس از الگوی Singleton (از طریق object) استفاده می‌کند تا همیشه یک نمونه از آن در کل اپلیکیشن در دسترس باشد.

متد startService(context: Context) وظایف (WorkRequest) مربوط به تمام تست‌ها (Ping, DNS, HTTP, Speed) را با استفاده از WorkManager زمان‌بندی می‌کند. این وظایف به صورت PeriodicWorkRequest تعریف شده‌اند، یعنی به صورت دوره‌ای (مثلاً هر ۶ ساعت) اجرا می‌شوند.

متد stopService(context: Context) تمام وظایف در حال اجرا و زمان‌بندی شده توسط WorkManager را لغو می‌کند.

- شبه‌کد:

```
1 OBJECT ServiceManager:
2   FUNCTION startService(context):
3
4     // Make periodic requests for each test
5     pingWorkRequest = PeriodicWorkRequest for PingTestWorker (repeat
every 6 hours)
6     dnsWorkRequest = PeriodicWorkRequest for DnsTestWorker (repeat
every 6 hours)
7     // ... and so on for other tests
8
9     // Placing tasks on WorkManager queue
10    WorkManager.getInstance(context).enqueueUniquePeriodicWork(
11    "pingTest",
```

```

12     ExistingPeriodicWorkPolicy.KEEP,
13     pingWorkRequest
14 )
15 // ... and so on for other tests
16
17 FUNCTION stopService(context):
18 // Kill all tasks
19 WorkManager.getInstance(context).cancelAllWork()
20

```

۲.۱.۳ فایل‌های *Worker.kt (مانند PingTestWorker.kt)

- هدف: هر کدام از این فایل‌ها یک CoroutineWorker هستند که مسئولیت اجرای یک نوع تست خاص را در پس‌زمینه دارند.

- توضیح کد (PingTestWorker.kt به عنوان مثال): کلاس PingTestWorker از CoroutineWorker ارث‌بری می‌کند. منطق اصلی در متد doWork() پیاده‌سازی می‌شود. درون doWork() یک شیء از NetworkTestService ساخته می‌شود. سپس متد executePingTest() از این سرویس فراخوانی می‌شود. این متد آدرس سرور هدف را از ورودی‌های Worker دریافت می‌کند. پس از اجرای تست، نتیجه (که شامل میانگین تأخیر، درصد بسته‌های از دست رفته و...) در یک شیء مدل (PingTestResult) ذخیره می‌شود. در نهایت، این نتیجه با استفاده از DatabaseHelper در پایگاه داده محلی SQLite ذخیره می‌شود. اگر عملیات موفقیت‌آمیز باشد، Result.success() و در غیر این صورت Result.failure() بازگردانده می‌شود.

● الگوریتم: Ping

۱. اجرای دستور ping سیستم عامل با پارامترهای مشخص (تعداد بسته‌ها، آدرس سرور).
۲. خواندن خروجی استاندارد (stdout) دستور.
۳. پارس کردن خروجی برای استخراج مقادیر آماری مانند حداقل، میانگین، حداکثر تأخیر و درصد packet loss.
۴. ذخیره این مقادیر.

● شبه‌کد:

```

1 CLASS PingTestWorker INHERITS CoroutineWorker:
2 FUNCTION doWork():
3 // Get input data (Like server address)
4 targetHost = inputData.getString("target")
5
6 // Test execution
7 networkTester = NetworkTestService()
8 pingResult = networkTester.executePingTest(targetHost)
9
10 // Save output in DB
11 dbHelper = DatabaseHelper(applicationContext)
12 dbHelper.insertPingResult(pingResult)
13
14 RETURN Result.success()
15

```

۳.۱.۳ فایل LocationService.kt و CellularService.kt

- هدف: این کلاس‌ها مسئول جمع‌آوری داده‌های جانبی اما بسیار مهم هستند: موقعیت مکانی و اطلاعات شبکه سلولی.
- توضیح کد LocationService.kt: این کلاس از FusedLocationProviderClient گوگل برای دریافت آخرین موقعیت مکانی شناخته‌شده دستگاه استفاده می‌کند. این روش بهینه‌تر از درخواست GPS جدید است و در مصرف باتری صرفه‌جویی می‌کند. متد getLastKnownLocation() یک suspendCancellableCoroutine را باز می‌گرداند که به صورت آسنکرون منتظر پاسخ از سرویس موقعیت‌یابی می‌ماند و در نهایت یک شیء Location را برمی‌گرداند.
- توضیح کد CellularService.kt: این کلاس از TelephonyManager اندروید برای استخراج اطلاعات شبکه موبایل استفاده می‌کند. اطلاعاتی مانند نام اپراتور (networkOperatorName)، نوع شبکه (networkType) مانند LTE, HSPA و قدرت سیگنال (SignalStrength) از طریق API های مربوطه خوانده می‌شوند. این اطلاعات به درک بهتر شرایط تست کمک می‌کنند.

۲.۳ پوشه views - رابط کاربری و منطق نمایش

این بسته شامل Activity ها و Fragment هایی است که رابط کاربری اپلیکیشن را تشکیل می‌دهند.

۱.۲.۳ فایل MainActivity.kt

- هدف: این اکتیویتی، صفحه اصلی اپلیکیشن پس از ورود کاربر است. این اکتیویتی به عنوان میزبان (Host) برای فرگمنت‌های اصلی (Home، Dashboard، Notifications) عمل می‌کند.
- توضیح کد: در متد onCreate()، این اکتیویتی binding را برای لایه activity_main.xml تنظیم می‌کند. یک NavController برای مدیریت ناوبری بین فرگمنت‌ها از طریق NavHostFragment پیدا می‌شود. BottomNavigationView و NavigationView (برای منوی کشویی) با NavController ست می‌شوند تا با انتخاب هر آیتم، فرگمنت مربوطه نمایش داده شود. همچنین منطق مربوط به باز و بسته شدن منوی کشویی (Drawer) در اینجا پیاده‌سازی شده است.

۲.۲.۳ فایل MapActivity.kt

- هدف: نمایش نقشه و داده‌های جمع‌آوری شده روی آن.
- توضیح کد: این اکتیویتی OnMapReadyCallback را پیاده‌سازی می‌کند. در onMapReady()، پس از آماده شدن نقشه، استایل نقشه (مثلاً حالت تاریک) بر اساس تم اپلیکیشن تنظیم می‌شود. یک MapDataSender برای دریافت داده‌ها از سرور (یا دیتابیس محلی) فراخوانی می‌شود. به ازای هر نقطه داده دریافت شده، یک Marker با استفاده از googleMap.addMarker() به نقشه اضافه می‌شود. موقعیت مارکر بر اساس طول و عرض جغرافیایی داده تنظیم می‌شود. یک CustomInfoWindowAdapter برای نمایش یک پنجره اطلاعاتی سفارشی هنگام کلیک روی هر مارکر تنظیم می‌شود. این پنجره جزئیات تست (مانند سرعت دانلود، پینگ و...) را نمایش می‌دهد.
- شبه کد:

```

1 CLASS MapActivity IMPLEMENTS OnMapReadyCallback:
2 FUNCTION onCreate():
3 // Get and Initialize map fragment
4 mapFragment.getMapAsync(this)
5
6 FUNCTION onMapReady(googleMap):
7 self.googleMap = googleMap
8
9 // Setting map style
10 setMapStyleBasedOnTheme()
11
12 // Fetch data
13 dataPoints = fetchDataFromServer()
14
15 // Add markers to the map
16 FOR each point IN dataPoints:
17 markerOptions = new MarkerOptions()
18 .position(new LatLng(point.latitude, point.longitude))
19 .title("Test Result")
20 .snippet("Details...")
21
22 googleMap.addMarker(markerOptions)
23
24 // Setting the customized InfoWindow
25 googleMap.setInfoWindowAdapter(new CustomInfoWindowAdapter())
26

```

۳.۲.۳ فایل‌های فرگمنت تست (مانند PingTestFragment.kt)

- هدف: این فرگمنت‌ها رابط کاربری لازم برای اجرای دستی (فعال) تست‌ها را فراهم می‌کنند.
- توضیح کد (PingTestFragment.kt به عنوان مثال): این فرگمنت یک EditText برای دریافت آدرس سرور هدف و یک Button برای شروع تست دارد. با کلیک روی دکمه، یک OneTimeWorkRequest برای PingTestWorker ساخته می‌شود. آدرس وارد شده توسط کاربر به عنوان داده ورودی به این Worker می‌شود. وظیفه در صف WorkManager قرار می‌گیرد. فرگمنت از طریق WorkManager.getInfoByTagLiveData() وضعیت اجرای وظیفه را مشاهده (Observe) می‌کند. وقتی وضعیت به SUCCEEDED تغییر می‌کند، فرگمنت نتیجه را از پایگاه داده محلی می‌خواند و آن را در یک RecyclerView نمایش می‌دهد.

۳.۳ پوشه utils - ابزارهای کمکی

این بسته شامل کلاس‌هایی است که وظایف عمومی و مشترک را در سراسر اپلیکیشن انجام می‌دهند.

۱.۳.۳ DatabaseHelper.kt فایل

- هدف: مدیریت کامل پایگاه داده محلی SQLite.
- توضیح کد: این کلاس از SQLiteOpenHelper ارث‌بری می‌کند. در متد onCreate()، جداول پایگاه داده (مانند ping_results، dns_results و غیره) با دستورات SQL ایجاد می‌شوند. هر جدول ستون‌های متناسب با داده‌های آن تست را دارد (مثلاً average_latency برای تست پینگ). در متد onUpgrade()، منطق مربوط به به‌روزرسانی شمای پایگاه داده در صورت تغییر نسخه پیاده‌سازی می‌شود.

این کلاس متدهایی برای تمام عملیات (Create, Read, Update, Delete) CRUD فراهم می‌کند. مثلاً `insertPingResult()`، `getAllPingResults()` و این متدها باعث می‌شوند بقیه کدها نیازی به درگیری مستقیم با دستورات SQL نداشته باشند.

۲.۳.۳ فایل `TokenManager.kt`

- هدف: مدیریت ذخیره و بازیابی توکن احراز هویت کاربر.
- توضیح کد: این کلاس از `SharedPreferences` برای ذخیره توکن استفاده می‌کند. `SharedPreferences` یک راه ساده برای ذخیره داده‌های کلید-مقدار (Key-Value) است. متد `saveToken(token)` توکن را در `SharedPreferences` ذخیره می‌کند. متد `getToken()` توکن ذخیره شده را بازیابی می‌کند. این کلاس باعث می‌شود که کاربر پس از یک بار ورود، تا زمانی که توکن معتبر است، نیازی به ورود مجدد نداشته باشد.

۴.۳ پوشه `models` - ساختارهای داده

این بسته شامل کلاس‌های داده (class data) است که ساختار اطلاعاتی را که در اپلیکیشن جابجا می‌شود (نتایج تست‌ها، اطلاعات کاربر و...) را تعریف می‌کنند. برای مثال:

- `PingTestResult.kt`: شامل فیلدهایی مانند `timestamp`، `targetHost`، `avgLatency`، `packetLoss` و

- `DownloadTestResult.kt`: شامل فیلدهایی مانند `timestamp`، `serverUrl`، `downloadSpeedMbps` و

استفاده از class data در کاتلین، پیاده‌سازی متدهایی مانند `equals()`، `hashCode()` و `toString()` را به صورت خودکار انجام می‌دهد و کد را خواناتر می‌کند.